

Extending Datalog for Matchmaking in P2P E-Marketplaces

Azzurra Ragone¹, Umberto Straccia² Tommaso Di Noia¹,
Eugenio Di Sciascio¹, Francesco M. Donini³

¹ Politecnico di Bari, Via Re David, 200, I-70125, Bari, Italy
{a.ragone,t.dinoia,disciascio}@poliba.it

² ISTI - CNR, Via G. Moruzzi 1, 56124 Pisa, Italy
straccia@isti.cnr.it

³ Università della Tuscia, via San Carlo, 32, I-01100, Viterbo, Italy
donini@unitus.it

Abstract. We present an approach to matchmaking in P2P e-marketplaces, which mixes in a formal and principled way Datalog, fuzzy sets and utility theory, in order to determine most promising matches between perspective counterparts.

Use of Datalog ensures the scalability of our approach to large marketplaces, while Fuzzy Logic provides a neat connection with logical specifications and allows to model soft constraints and *how well* they could be satisfied by an agreement. Noteworthy is that our approach takes into account in the peer-to-peer matchmaking also preferences of each counterpart and their utilities. This allows to rule out of the match list those counteroffers that, although seemingly appealing for the buyer, would probably lead to failure due to contrasting preferences of the seller, and paves the way to the actual negotiation stage.

1 Introduction

In e-commerce settings, matchmaking can be defined as the process of finding “good” counterparts for a given entry in an marketplace. While in e-marketplaces dealing with undifferentiated products (as commodities) the “best” counterpart is the one offering the “best” price per unit, in e-marketplace dealing with complex products —cars, houses, computers— the price is not the single feature to take into account. Also, ads can involve bundle of issues, *e.g.*, *Sports car with optional package including both GPS system and alarm system* or implications *e.g.*, *If a car has leather seats then it is also provided with air conditioning*, and some kind of logical theory, able to let users express their needs/offers, could surely help. A matchmaking system has to be able both to find “good” counterparts and to evaluate how “good” a counterpart is. Currently, many commercial sites force the buyer to enter her request browsing a predefined classification that may be completely unsuitable for the characteristics the buyer might have in mind *e.g.*, they require to enter a brand first, then a model of that brand, etc. while a buyer may be not interested in a specific brand, but only on some limitations on price and color. In this respect, one may say that they provide no matchmaking assistance: the matchmaker is the buyer herself.

To assist buyers and sellers in marketplaces, several research proposals on matchmaking systems were issued. They either try to compute a score of possible counterparts, based on textual information [24], or to compare the logical representations of supply and demand [9], or combine both scores and logic in some way [6, 14]. Our proposal falls in this last category, mixing in a formal way Datalog, Fuzzy sets, and Utility Theory. While the above

logic-based proposals do not tackle the scalability problem, our resort on Datalog ensures the scalability of our approach to large marketplaces. On the other hand, the resort on Fuzzy Logic ensures a neat connection with the logical specification, while allowing the system to give an explanation of suggestions in terms of how well the preferences could be satisfied by an agreement.

Since we want both buyer and seller equally satisfied, the matchmaker computes a score as the maximum value of the *product* of the utilities of the buyer u_β and the seller u_σ over all possible agreements. In this way both buyer's and seller's preferences are taken into account ruling out of the match list those counteroffers that, although seemingly appealing for the buyer, would probably lead to failure due to contrasting preferences of the seller. The remaining of the paper is as follow: to set the stage we introduce basics of Datalog^{topk} and requirements for matchmaking process. Then fuzzy soft constraints are presented followed by the description of fuzzy matchmaking process in Datalog^{topk}. Discussion about relevant related work and conclusions close the paper.

2 Top-k Datalog

We use an extension of *Datalog* (cf. [1, 5, 23]) as our representation and query tool. We extend it by allowing soft constraint predicates to appear in rules and queries (we call the language Datalog^{topk}). The proposal here extends the work [20] in which soft constraint predicates, i.e., fuzzy predicates, may appear in a query rules only and is conceptually equivalent to [21]. Basically, we allow vague/fuzzy predicates to occur in rule bodies, which have the effect that each tuple in the answer set of a query has now a score in [0,1]. Datalog^{topk} addresses the problem to compute the top-k answers in case the set of facts is huge, without evaluating all the tuples' score. As matching a buyer's request with a seller's offer is a matter of degree, we will use Datalog^{topk} to find the top-k matchings. Datalog^{topk} is as Datalog except that we additionally allow fuzzy predicates to occur in Datalog rule bodies. Specifically, let r be an $n + 1$ -ary predicate. A Datalog^{topk} rule is of the form

$$r(\mathbf{x}, s) \leftarrow \exists y \text{body}(\mathbf{x}, \mathbf{y}), s = f(p_1(\mathbf{z}_1), \dots, p_n(\mathbf{z}_n))$$

where

1. $\mathbf{x}$ are the n distinguished variables;
2. s is the score variable, taking values in $[0, 1]$, and r is functional on s ;
3. $\mathbf{y}$ are so-called non-distinguished variables and are distinct from the variables in $\mathbf{x}$;
4. $\text{body}(\mathbf{x}, \mathbf{y})$ is a conjunction of Datalog atoms;
5. $\mathbf{z}_i$ are tuples of constants or variables in $\mathbf{x}$ or $\mathbf{y}$;
6. p_i is an n_i -ary fuzzy predicate assigning to each n_i -ary tuple $\mathbf{c}_i$ as score $p_i(\mathbf{c}_i) \in [0, 1]$;
7. f is a scoring function $f: [0, 1]^n \rightarrow [0, 1]$, which combines the scores of the n fuzzy predicates p_i into an overall query score to be assigned to the score variable s . We assume that f is monotone, i.e., for each $\mathbf{v}, \mathbf{v}' \in [0, 1]^n$ such that $\mathbf{v} \leq \mathbf{v}'$, $f(\mathbf{v}) \leq f(\mathbf{v}')$ holds, where $(v_1, \dots, v_n) \leq (v'_1, \dots, v'_n)$ iff $v_i \leq v'_i$ for all i ;
8. We assume that the computational cost of f and all fuzzy predicates p_i is bounded by a constant.

We call $s = f(p_1(\mathbf{z}_1), \dots, p_n(\mathbf{z}_n))$ a scoring atom. For instance,

$$\begin{aligned} \text{CheapCar}(x, p, s) &\leftarrow \text{NewCar}(x), \text{CarPrice}(x, p), s = \max(0, 1 - p/15000) \\ \text{CheapCar}(x, p, s) &\leftarrow \text{SecondHandCar}(x), \text{CarPrice}(x, p), s = \max(0, 1 - p/7500) \end{aligned}$$

are two Datalog topk rules looking for cheap cars, assigning to each car a score depending on its price. If the price of a new car is above 15000 the car is not considered as a cheap one, while the scoring function is increasing as the price lowers. Hence, it is quite natural that if we are looking for cheap cars one wants that the retrieved cars are sorted in decreasing order with respect its score, *i.e.*, degree of cheapness. Furthermore, as the database may contain thousands of tuples, one usually wants to retrieve just the top- k ranked ones.

A Datalog topk query program is a pair $\mathcal{P} = \langle \mathcal{P}', q \rangle$, where $\mathcal{P}'$ is Datalog topk program, q is a query rule and q does not occur in $\mathcal{P}'$. Essentially, the difference between Datalog and Datalog topk is that now scoring atoms may appear in the rule body of “query rules”.

The basic inference services that concerns us is the top- k retrieval problem, where this latter is defined as:

Top- k retrieval: Given a Datalog topk program $\mathcal{P}$, retrieve the top- k ranked tuples $\langle \mathbf{c}, v \rangle$ that instantiate the query q and rank them in decreasing order w.r.t. the score v , *i.e.*, find the top- k ranked tuples of the answer set of q , denoted

$$ans_k(\mathcal{P}, q) = \text{Top}_k \{ \langle \mathbf{c}, v \rangle \mid \mathcal{P} \models q(\mathbf{c}, v) \} .$$

For instance,

$$q(x, p, s) \leftarrow \text{CheapCar}(x, p, s)$$

is a query asking for cheap cars. The top- k ranked cars, according to the score (that depends on their price), is obtained by $ans_k(\mathcal{P}, q)$.

3 Matchmaking Requirements

In the whole paper we adopt as reference in examples a P2P automobile marketplace, and motivate our work in this domain. With respect to this e-marketplace the users can express different preferences on several features, *i.e.*, a buyer can specify conditional preferences, such as *if the car is a sports one, then the feeding has to be gasoline*, or *a cheap car, yet if the car is provided with a GPS system she is ready to pay up to 17000*.

Looking at the previous example, we notice the buyer’s requests can be split into two different parts. In fact, the buyer expresses as a *hard* requirement that if the car is a sports one then the feeding has to be gasoline and as a preference, *soft* constraint, that she is willing to buy a cheap car but she could spend some more if the car is equipped with a GPS system. Strict requirements represent what the buyer and the seller want to be necessarily satisfied in order to accept the final agreement – in our framework we call strict requirements *hard constraints*. Preferences denote issues they are willing to negotiate on – this is what we call *soft constraints*. Let us now introduce an example request, we will use to explain some aspects of our approach:

Example 1. Suppose to have a buyer’s request like “*I want a station wagon black or gray. Preferably I would like to pay less than 14,000 € furthermore I’m willing to pay up to 17,000 € if warranty is greater or equal than 100000 km. (I don’t want to pay more than 19,000 € and I don’t want a car with a warranty less than 60,000 km)*”. In this example we identify:

hard constraints = Body Type: Station wagon. **Color:** Black or Grey. **Price:** $\leq 19,000$ €. **Warranty:** $\geq 60,000$ km.

soft constraints = Price: $\leq 14,000$. **Warranty-Price:** if Warranty $\geq 100,000$ then Price $\leq 17,000$ €

3.1 Preferences and Utilities

In a matchmaking process, retrieving supplies matching the request taking into account only *hard constraints* would be trivial. The matchmaker should just look for agreements where *hard constraints* of both the buyer and seller are satisfied. Given a request and a set of retrieved supplies, how should the matchmaker find –and rank– the most *suitable* or *promising* agreements to propose to both parties? The matchmaker should exploit *soft constraints* expressed both by the buyer and the seller.

Among the supplies completely satisfying all the requirements modeled in *hard constraints*, the top-ranked ones will be those best satisfying features expressed in the *soft constraints* proposed both by the buyer and the seller.

For what concerns *soft constraints*, we observe that the buyer's and seller's satisfaction degree, with respect to a final agreement, depends on which parts of her preference specifications have been satisfied. For instance, w.r.t. Example 1 suppose to have the following two supplies, where only soft constraints have been specified:

σ' : **Body Type:** Station wagon. **Color:** Grey. **Price:** 16,000 €. **Warranty:** 200,000 km.
 σ'' : **Body Type:** Station wagon. **Color:** Black **Price:** 13,000 €. **Warranty:** 50,000 km

Comparing these supplies with buyer's *soft constraints* we note that σ' satisfies the preference $\beta_2 = \{\mathbf{Warranty-Price: if Warranty} \geq 100,000 \text{ then Price} \leq 17,000 \text{ €}\}$, while σ'' the preference $\beta_1 = \{\mathbf{Price:} \leq 14,000\}$. How to evaluate the best one? We expect the buyer assigns a positive utility value representing the preference relevance to sub-parts of *soft constraints*. In this case we assume utility values — $u(\beta_1)$ and $u(\beta_2)$ — both for β_1 and β_2 . Actually, the same holds from the seller's perspective. In fact, in a P2P e-marketplace the seller may express his preferences — *soft constraints e.g.*, on selling price, warranty, delivery time — with corresponding utilities $u(\sigma_j)$, as well as his *hard constraints* (e.g., color, model, engine fuel, etc.).

The only constraint on utility values is that both seller's and buyer's ones are normalized to 1 to eliminate outliers, and make them comparable [11].

$$\sum u(\beta_i) = 1 \qquad \sum u(\sigma_j) = 1 \qquad (1)$$

Since we assume utilities on preferences as additive, here we can write the global utility of the buyer u_β and of the seller u_σ as just a sum of the utilities of preferences satisfied in the agreement. If we are able, for each preference β_i of the buyer and σ_j of the seller, to evaluate a score s_i and s_j representing a degree of preference satisfaction, then it is reasonable to think that the global utility has to take into account also these information. Hence, in formulas, we write the two utility functions as:

$$u_\beta = \sum s_i * u(\beta_i) \qquad u_\sigma = \sum s_j * u(\sigma_j) \qquad (2)$$

From Example 1, we note that while considering numerical features, it is still possible to express hard and soft constraints on them. A hard constraint on a numerical feature can be considered as a **reservation value** [18] on the feature itself.

3.2 The Matchmaking Process

Based on the notions of preferences, utility, and reservation values introduced so far, we begin outlining the actual matchmaking process. Every time a seller (buyer) enters the marketplace, he proposes his supply (request) expressing both *hard constraints* and *soft constraints* (preferences). Then, he sets his reservation value $r_{\sigma,f}$ on numerical feature f . Eventually,

for each preference $\sigma_j(\beta_i)$ he expresses the corresponding utility $u(\sigma_j)$ ($u(\beta_i)$). Based on buyer's and seller's specifications, the matchmaker returns a ranked list of supplies such that: [a] they satisfy both the *hard constraints* in the request and conversely their *hard constraints* are satisfied by the request; [b] the rank is evaluated taking into account preferences and utility functions u_β and u_σ as defined by equations (2). In a P2P e-marketplace, usually the aim is to find agreements maximizing not only the buyer's satisfaction or the seller's one, but trying to make them equally satisfied. So the matchmaker has to propose agreements mutually beneficial for both of them. Such agreements are computed considering the higher values of u_β and u_σ **utilities product** [16].

4 Requirements Fuzzy Representation

Marketplaces are typical scenarios where the notion of fuzziness is often involved. The concept of *Cheap* or *Expensive* are quite usual. Similarly, numerical variables involved in a commercial transaction expose a fuzzy behavior. For instance, suppose to have a buyer looking for a car provided with a warranty greater than 100,000 kilometers and a supplier selling his car with a 80,000 kilometers warranty. We can not say they do not match at all. Instead we can say they match with a certain degree. A logical language able to deal with fuzzy information would be then a good choice to model matchmaking. To represent buyer/seller requirements in a Datalog^{topk} setting, hereafter we will use the following notation:

$$\begin{aligned} \text{hard constraints} &= \begin{cases} \beta(x, \mathbf{y}) \text{ or } \beta(x) , \text{ buyer's strict requirements} \\ \sigma(x, \mathbf{y}) \text{ or } \sigma(x) , \text{ seller's strict requirements} \end{cases} \\ \text{soft constraints} &= \begin{cases} \beta_i(x, \mathbf{y}, s) \text{ or } \beta_i(x, s) , \text{ buyer's preferences} \\ \sigma_j(x, \mathbf{y}, s) \text{ or } \sigma_j(x, s) , \text{ seller's preferences} \end{cases} \end{aligned}$$

where x is a single variable, $\mathbf{y}$ (if present) represents a vector of numerical variables and s represents the score variable as defined in Section 2. Since a score is associated to each fuzzy predicate, we can compute the global utility based on the two utility functions in Section 3.1. The two predicates σ and β model the minimal requirements the buyer and the seller want to be satisfied in order to accept the final agreement. Notice that, if seller and buyer set hard constraints in conflict with each other, the corresponding supply will not be retrieved and no agreement will be reached. *Soft constraints* are modeled via Datalog predicates β_i for the buyer and σ_j for the seller, where each of them represents a sub-part of the buyer/seller preferences.

The use of x , y and s should be clearer looking at how buyer's request in Example 1 is formalized:

$$\begin{aligned}
\beta_A(x) &\leftarrow StationWagon(x) \\
\beta_B(x, p) &\leftarrow CarPrice(x, p), 0 \leq p \leq 19000 \\
\beta_C(x, kmw) &\leftarrow KmWarranty(x, kmw), kmw \geq 60000 \\
\beta_D(x) &\leftarrow Grey(x) \\
\beta_D(x) &\leftarrow Black(x) \\
\beta(x, p, kmw) &\leftarrow \beta_A(x), \beta_B(x, p), \beta_C(x, kmw), \beta_D(x)
\end{aligned}$$

$$\begin{aligned}
\beta_1(x, p, s) &\leftarrow CarPrice(x, p), \\
&LS(0, 100000, 14000, 16000, p, s)
\end{aligned}$$

$$\begin{aligned}
\beta_2(x, p, kmw, s) &\leftarrow KmWarranty(x, kmw), CarPrice(x, p) \\
&RS(0, 400000, 80000, 100000, kmw, s_1), \\
&LS(0, 100000, 17000, 19000, p, s_2), \\
&s = \max(1 - s_1, s_2)
\end{aligned}$$

5 Fuzzy Matchmaking in Datalog^{topk}

Now we have all what is needed to model the matchmaking framework in a Datalog^{topk} setting. First of all we show how to write the corresponding Datalog^{topk} program $\mathcal{P}_{match} = \langle \mathcal{P}_I, \mathcal{P}_E \rangle$.

1. for each supply write the corresponding Datalog fact and add it to the Extensional Database $\mathcal{P}_E$;
2. encode buyer's *hard constraints* requirements as a Datalog rule where the head contains the predicate $\beta(x, y)$ as shown in Section 4; add the rule to the Intensional Database $\mathcal{P}_I$;
3. encode seller's *hard constraints* requirements as a Datalog rule where the head contains the predicate $\sigma(x, y)$; add the rule to $\mathcal{P}_I$;
4. for each buyer's preference β_i , write the corresponding rule in Datalog^{topk} where the head contains the predicate $\beta_i(x, y, s)$ as shown in Section 4; add the rule to $\mathcal{P}_I$; set the utility value $u(\beta_i)$ as shown in Section 3.1;
5. for each seller's preference σ_j , write the corresponding rule in Datalog^{topk} where the head contains the predicate $\sigma_j(x, y, s)$ as shown in Section 4; add the rule to $\mathcal{P}_I$; set the utility value $u(\sigma_j)$ as shown in Section 3.1;
6. add to $\mathcal{P}_I$ the rules:

$$\begin{aligned}
Buyer(x, y, u_\beta) &\leftarrow \beta(x, y_0), \beta_1(x, y_1, s_1), \beta_2(x, y_2, s_2), \dots, \\
&u_\beta = u(\beta_1) \cdot s_1 + u(\beta_2) \cdot s_2 + \dots
\end{aligned} \tag{3}$$

$$\begin{aligned}
Seller(x, y, u_\sigma) &\leftarrow \sigma(x, y_0), \sigma_1(x, y_1, s_1), \sigma_2(x, y_2, s_2), \dots, \\
&u_\sigma = u(\sigma_1) \cdot s_1 + u(\sigma_2) \cdot s_2 + \dots
\end{aligned} \tag{4}$$

where for each variable in y in the head of one of the two previous rules, the same variable occurs in at least one of the arrays of the corresponding body: $y_0, y_1, y_2, \dots$;

7. add to $\mathcal{P}_I$ the "query rule":

$$\begin{aligned}
Match(x, y, u) &\leftarrow Buyer(x, y_\beta, u_\beta), Seller(x, y_\sigma, u_\sigma), \\
&u = u_\beta \cdot u_\sigma
\end{aligned} \tag{5}$$

where for each variable in y in the head of the rule, the same variable occurs in y_β or y_σ

Once we have the Datalog topk program $\mathcal{P}_{match}$, then solve the following **Top- k retrieval** problem:

$$ans_k(\mathcal{P}_{match}, Match) = \text{Top}_k\{\langle x, \mathbf{y}, u \rangle \mid \langle \mathbf{y}, u \rangle \in \text{Top}_1\{\langle x, \mathbf{y}', u' \rangle \mid \mathcal{P} \models Match(x, \mathbf{y}', u')\}\}.$$

Basically, for each key value x of the database, we compute the best matching $\langle \mathbf{y}, u \rangle$ for it, i.e., $\langle \mathbf{y}, u \rangle \in \text{Top}_1\{\langle x, \mathbf{y}', u' \rangle \mid \mathcal{P} \models Match(x, \mathbf{y}', u')\}$, and then rank the top- k key values.

Notice that the rank is computed considering the product of buyer's and seller's utilities as stated at the end of Section 3.2.

6 Related Work and Discussion

In this paper we presented a fuzzy matchmaking approach exploiting Datalog to find the most promising agreements in a P2P e-marketplace. More precisely, exploiting fuzzy rules we have been able to model *soft constraints*, while taking into account both buyer's and seller's preferences and utilities to find agreements mutually beneficial for them both. The P2P matchmaking process is modeled as a Datalog program able also to consider background domain knowledge while keeping the approach effective and scalable. A prototype is currently being implemented to further validate the approach through large scale experiments. Recently, the problem of matchmaking has been investigated under different perspectives and many approaches have been proposed. An initial approach to matchmaking can be dated back to vague query answering [15] where the need to go beyond pure relational databases was addressed using weights attributed to several search variables. More recently similar approaches have been proposed extending SQL with “preference” clauses, in order to allow relaxed queries in structured databases [10] where only buyer's preferences are taken into account while retrieving promising supplies. Classified-ads matchmaking, at a syntactic level, was proposed in [19] and [24] to perform a matchmaking between semi-structured descriptions. Approaches to matchmaking using LOOM as description language can be found, among others, in [3] and [8]. Due to the growing interest in the Semantic Web initiative many approaches to matchmaking have been proposed based on Description Logics (DL). Matchmaking as satisfiability of concept conjunction in DLs was first proposed in [9]. In the framework of Retsina Multiagent infrastructure [22], a specific language was defined for agent advertisement, and matchmaking engine was developed [17], which carries out the process on five possible match levels. This approach was later extended in [13], with two new levels for matching classification. A similar classification was proposed — in the same venue — in [7], along with properties that a matchmaker should have in a DL based framework, and algorithms to classify and semantically rank matches within classes. An initial DL-based approach, adopting penalty functions ranking, has been proposed in [4], in the framework of dating systems. An extended matchmaking approach, with negotiable and strict constraints in a DL framework has been proposed in [6], using both concept contraction and concept abduction. Approximation and ranking in DL-based approaches to matchmaking has also recently led to adopting fuzzy-DLs, as in sMART [2] or hybrid approaches, as in the OWLS-MX matchmaker [12]. In [14] a language able to express conditional preferences is proposed to perform a matchmaking in Description Logics. Also in this case nothing is said on how to compute a agreement — as needed in P2P scenarios. Furthermore, the notion of fuzzy/vague requirements is not addressed.

References

1. S. Abiteboul, R. Hull, and V. Vianu. *Foundations of Databases*. Addison Wesley Publ. Co., Reading, Massachusetts, 1995.
2. S. Agarwal and S. Lamparter. smart - a semantic matchmaking portal for electronic markets. In *Proceedings of the 7th International IEEE Conference on E-Commerce Technology*, 2005.
3. Y. Arens, C. A. Knoblock, and W. Shen. Query Reformulation for Dynamic Information Integration. 6:99–130, 1996.
4. A. Cali, D. Calvanese, S. Colucci, T. Di Noia, and F. M. Donini. A description logic based approach for matching user profiles. In *Proceedings of the 17th International Workshop on Description Logics (DL'04)*, volume 104 of *CEUR Workshop Proceedings*, 2004.
5. S. Ceri, G. Gottlob, and L. Tanca. *Logic Programming and Databases*. Springer Verlag, 1990.
6. S. Colucci, T. Di Noia, E. Di Sciascio, F. Donini, and M. Mongiello. Concept Abduction and Contraction for Semantic-based Discovery of Matches and Negotiation Spaces in an E-Marketplace. *Electronic Commerce Research and Applications*, 4(4):345–361, 2005.
7. T. Di Noia, E. Di Sciascio, F. M. Donini, and M. Mongiello. A system for principled matchmaking in an electronic marketplace. *International Journal of Electronic Commerce*, 8(4):9–37, 2004.
8. Y. Gil and S. Ramachandran. PHOSPHORUS: a Task based Agent Matchmaker. In *Proc. International Conference on Autonomous Agents '01*, pages 110–111. ACM, 2001.
9. J. Gonzales-Castillo, D. Trastour, and C. Bartolini. Description Logics for Matchmaking of Services. In *Proceedings of the KI-2001 Workshop on Applications of Description Logics (ADL-2001)*, volume 44. CEUR Workshop Proceedings, 2001.
10. B. Hafenrichter and W. Kießling. Optimization of relational preference queries. In *The Sixteenth Australasian Database Conference (ADC 2005)*, pages 175–184, Newcastle, Australia, Jan. 2005.
11. R. L. Keeney and H. Raiffa. *Decisions with multiple objectives - preferences and value trade-offs*. Cambridge University Press, 1993.
12. M. Klusch, B. Fries, M. Khalid, and K. Sycara. Owls-mx: Hybrid owl-s service matchmaking. In *Proceedings of 1st Intl. AAAI Fall Symposium on Agents and the Semantic Web*, 2005.
13. L. Li and I. Horrocks. A Software Framework for Matchmaking Based on Semantic Web Technology. In *Proc. of WWW '03*, 2003.
14. T. Lukasiewicz and J. Schellhase. Variable-strength conditional preferences for matchmaking in description logics. In *Proc. of KR 2006*, pages 164–174, 2006.
15. A. Motro. VAGUE: A User Interface to Relational Databases that Permits Vague Queries. *ACM Trans. Office Inf. Syst.*, 6(3):187–214, 1988.
16. J. F. Nash. The bargaining problem. *Econometrica*, 18 (2):155–162, 1950.
17. M. Paolucci, T. Kawamura, T. Payne, and K. Sycara. Semantic Matching of Web Services Capabilities. In *The Semantic Web - ISWC 2002*, number 2342, pages 333–347. 2002.
18. H. Raiffa, J. Richardson, and D. Metcalfe. *Negotiation Analysis - The Science and Art of Collaborative Decision Making*. The Belknap Press of Harvard University Press, 2002.
19. R. Raman, M. Livny, and M. Solomon. Matchmaking: distributed resource management for high throughput computing. In *Proceedings of IEEE High Performance Distributed Computing Conf.*, pages 140–146, 1998.
20. U. Straccia. Towards top-k query answering in deductive databases. In *Proceedings of the 2006 IEEE International Conference on Systems, Man and Cybernetics (SMC-06)*, pages 4873–4879. IEEE, 2006.
21. U. Straccia. Towards vague query answering in logic programming for logic-based information retrieval. In *World Congress of the International Fuzzy Systems Association (IFSA-07)*, Cancun, Mexico, 2007.
22. K. Sycara, M. Paolucci, M. Van Velsen, and J. Giampapa. The RETSINA MAS infrastructure. *Autonomous agents and multi-agent systems*, 7:29–48, 2003.
23. J. D. Ullman. *Principles of Database and Knowledge Base Systems*, volume 1,2. Computer Science Press, Potomac, Maryland, 1989.
24. D. Veit, J. Muller, M. Schneider, and B. Fiehn. Matchmaking for Autonomous Agents in Electronic Marketplaces. In *Proc. International Conference on Autonomous Agents '01*, pages 65–66. ACM, 2001.